

The Beta-binomial model

José G. Dias

Table of contents

1. The Bernoulli distribution	1
2. The binomial distribution	2
3. The maximum likelihood estimation of p (Bernoulli population)	3
4. Bayesian inference about p	5
5. The beta distribution	5
6. The beta-binomial model	7
7. Example for the Bernoulli population	7
8. Our very first example on sampling from the posterior distribution	10
9. The logit and probit transformations	16

This text covers the Bernoulli and Binomial distributions used to model binary outcomes. First, we define the model and its visualization. Then, we focus on the estimation of parameter p using ML and Bayesian statistics.

1. The Bernoulli distribution

The Bernoulli distribution is a discrete probability distribution with two possible outcomes, typically labeled as success (coded as 1) and failure (coded as 0). The probability mass function (pmf) of the Bernoulli distribution is given by:

$$P(X = x) = \begin{cases} p & \text{if } x = 1 \\ 1 - p & \text{if } x = 0 \end{cases}$$

where p is the probability of success, i.e., $p = P(X = 1)$, with $E[X] = p$ and $Var[X] = p(1 - p)$.

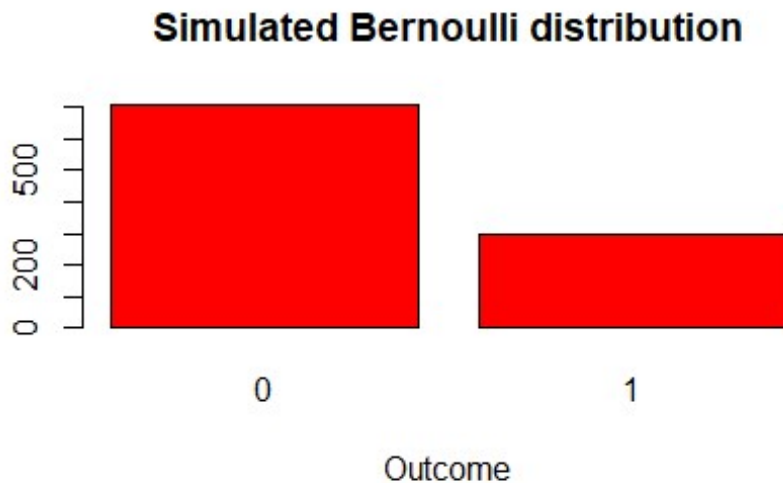
The next example simulates a sample of size 1000 from a Bernoulli population with $p = 0.3$.

Simulate Bernoulli distribution

```
set.seed(123)
p_success <- 0.3
bernoulli_data <- rbinom(1000, size = 1, prob = p_success)
```

```
# Plot simulated data
```

```
barplot(table(bernoulli_data), main = "Simulated Bernoulli distribution",  
xlab = "Outcome", col = "red", border = "black")
```



2. The binomial distribution

The binomial distribution models the number of successes in a fixed number of independent Bernoulli trials. The probability mass function (pmf) of a binomial distribution is given by:

$$P(X = x) = \binom{m}{x} p^x (1 - p)^{m-x}$$

where m is the number of independent trials, x is the number of successes, and p is the probability of success in a single trial. In the case of the binomial, we have $E[X] = mp$ and $Var[X] = mp(1 - p)$. For $m = 1$, it is the Bernoulli distribution.

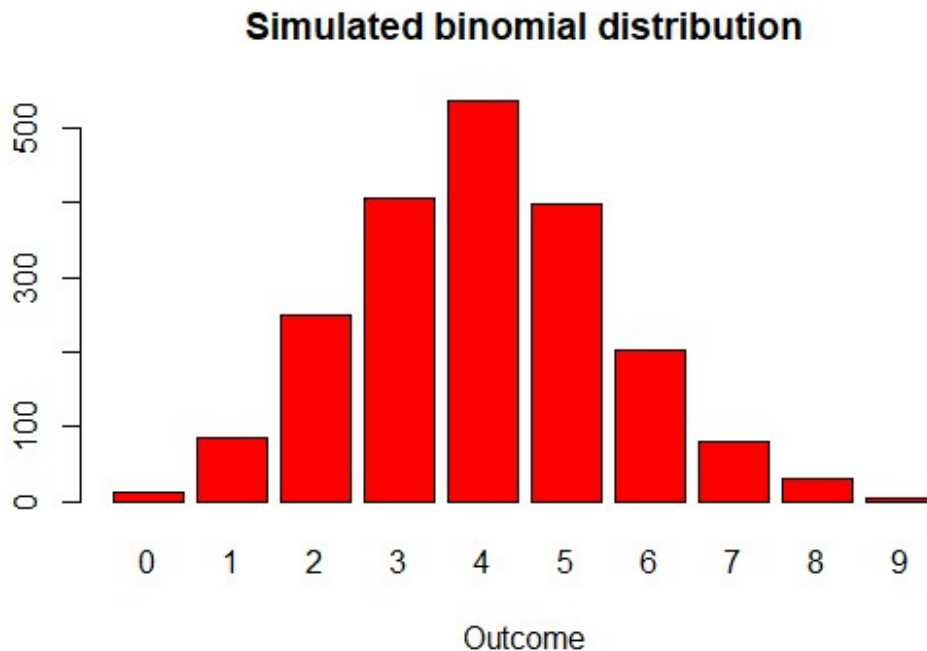
The next example simulates a sample of 2000 observations from a binomial population with $p = 0.4$ and $m = 10$.

```
# Simulate binomial distribution
```

```
set.seed(123)  
p_success <- 0.4  
n_trials <- 10  
binomial_data <- rbinom(2000, size = n_trials, prob = p_success)
```

```
# Plot simulated data
```

```
barplot(table(binomial_data), main = "Simulated binomial distribution", xlab
= "Outcome", col = "red", border = "black")
```



3. The maximum likelihood estimation of p (Bernoulli population)

The likelihood function for a single observation is given by the probability mass function (pmf) of the Bernoulli distribution. The likelihood function assumes a sample of n independent and identically distributed (i.i.d.) observations $(x_1, x_2, \dots, x_n)$ is:

$$L(p|x_1, \dots, x_n) = p^{x_1}(1-p)^{1-x_1} \times p^{x_2}(1-p)^{1-x_2} \times \dots \times p^{x_n}(1-p)^{1-x_n}$$

where x_i is the outcome of the i -th observation (0 for failure, 1 for success). The logarithm of the likelihood function (log-likelihood) is given by:

$$\ell(p|x_1, \dots, x_n) = \sum_{i=1}^n [x_i \log(p) + (1 - x_i) \log(1 - p)].$$

To find the maximum likelihood estimator (MLE) of p , we have to maximize $\ell(p|x_1, \dots, x_n)$ with respect to p . This can be done by taking the derivative of $\ell(p|x_1, \dots, x_n)$ with respect to p , setting it equal to zero, and solving for p . In practice, it is convenient to work with the log-likelihood.

Using the simulated sample from the Bernoulli population the (minus) log-likelihood function is depicted below.

```
# Define the (negative) log-likelihood function for a Bernoulli distribution
```

```
llk <- function(p, data) {
```

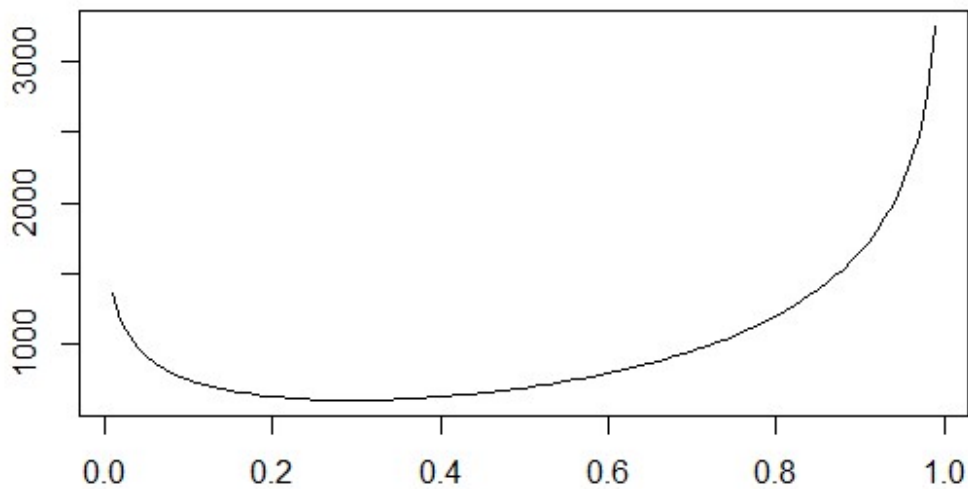
```

    -sum(data * log(p) + (1 - data) * log(1 - p))
  }

# Log likelihood curve

xx <- seq(0.01, 0.99, .01)
yy <- sapply(X=xx, FUN=llk, data =bernoulli_data)
plot(xx,yy,type='l', ylab='', xlab='')

```



```

# Maximize the log-likelihood to find MLE of p

mle_result <- optim(par = 0.3, fn = llk, data =
bernoulli_data,method="Brent",
                    lower = 0.000001, upper = 0.999999)

```

The next code obtains the minimum of this function that is the MLE:

```

# Maximize the log-likelihood to find MLE of p

mle_result <- optim(par = 0.3, fn = llk, data =
bernoulli_data,method="Brent",
                    lower = 0.000001, upper = 0.999999)
cat("p estimate (ML optimization):", mle_result$par, "\n")

p estimate (ML optimization): 0.295

```

We know the analytic (close-form) expression for the ML estimator of p using derivatives:

$$\hat{p} = \frac{1}{n} \sum_{i=1}^n X_i$$

that corresponds to the sample proportion.

Now we can compare these two results (not always a closed-form solution is available):

Compare these results

```
p_equation <- mean(bernoulli_data)
cat("p estimate (MLE proportion):", p_equation, "\n")

p estimate (MLE proportion): 0.295
```

4. Bayesian inference about p

Bayesian methods incorporate prior knowledge and uncertainty into the modeling process, which can be particularly valuable in situations with limited data or when prior knowledge is available.

The posterior distribution for p given x can still be described in terms of the likelihood and prior:

$$\pi(p|x) \propto \pi(x | p)\pi(p)$$

where $\pi(x | p)$ and $\pi(p)$ are the likelihood function and prior distribution, respectively. The prior distribution for p has support domain $[0,1]$. A convenient choice is the beta distribution, because it is a conjugate prior for the binomial likelihood, making it particularly convenient for Bayesian estimation. This beta-binomial model allows us to update our beliefs about the success probability p based on observed data.

5. The beta distribution

The beta distribution has a probability density function (pdf) given by:

$$f(p|\alpha, \beta) = \frac{p^{\alpha-1}(1-p)^{\beta-1}}{B(\alpha, \beta)}$$

where $B(\alpha, \beta) = \int_0^1 t^{\alpha-1} (1-t)^{\beta-1} dt$ is the beta function. The beta distribution is a continuous probability distribution defined on the interval $[0, 1]$. The parameters α and β , in Bayesian statistics called hyperparameters, control the shape of the distribution. For instance, for $\alpha = \beta = 1$, the beta distribution is equivalent to the uniform distribution on $[0, 1]$.

The graph below shows the beta distribution for different values of $\alpha = (1,3,5)$ and $\beta = (1,3,5)$, including the case of the uniform distribution.

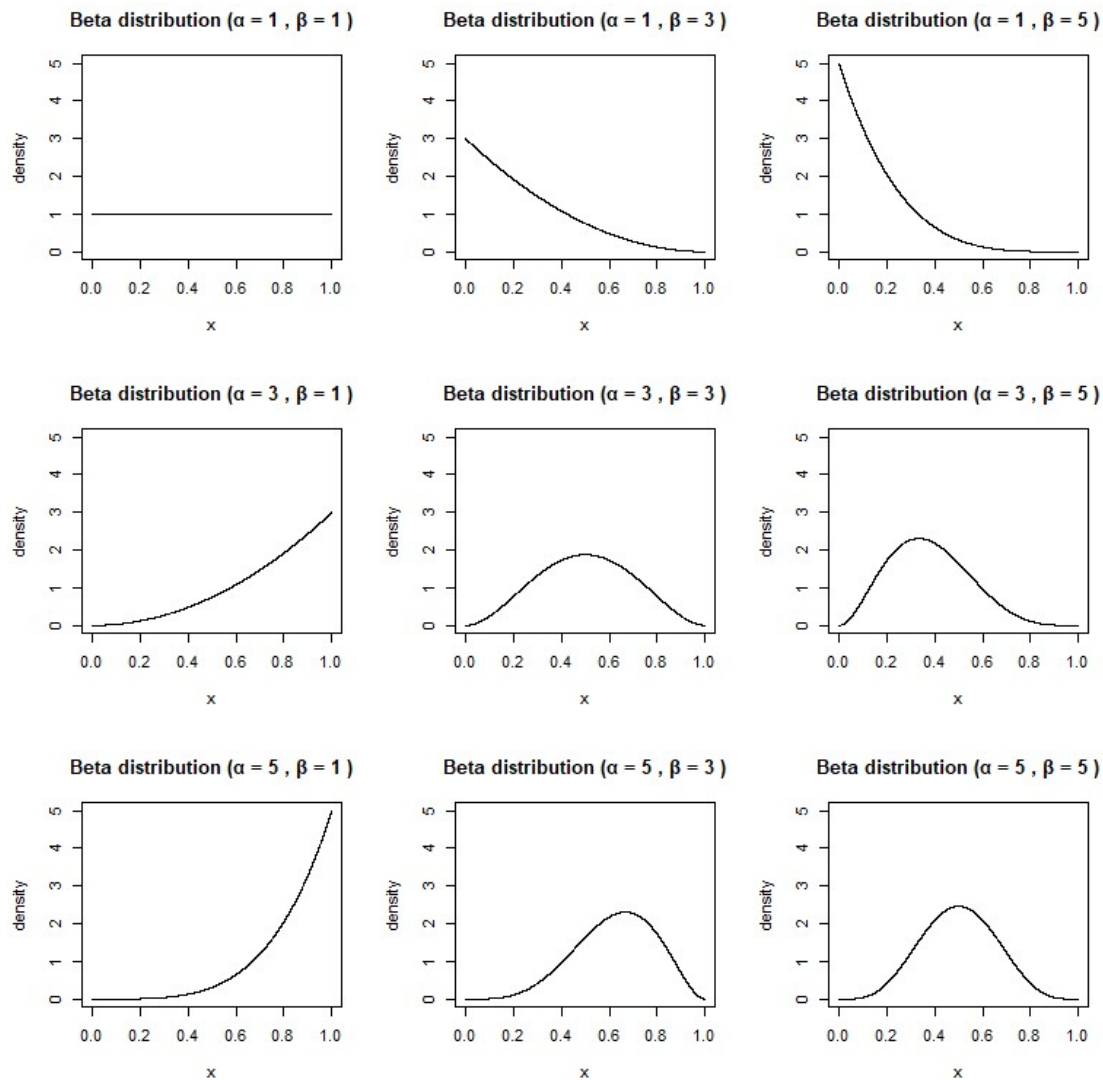
```

# Plot beta distribution for different parameter values
alpha_values <- c(1,3,5)
beta_values <- c(1,3,5)

par(mfrow = c(3, 3))

for (i in 1:length(alpha_values))
  for (j in 1:length(beta_values)){
    xx <- seq(0, 1, length.out = 5000)
    yy <- dbeta(xx, shape1 = alpha_values[i], shape2 = beta_values[j])
    tit <- paste("Beta distribution ( $\alpha =$ ", alpha_values[i], ",  $\beta =$ ",
beta_values[j], ")")
    plot(xx,yy,type='l',xlim = c(0, 1), ylim = c(0, 5),
        ylab='density',xlab='x',main = tit)
  }

```



```
# Reset plotting layout
par(mfrow = c(1, 1))
```

6. The beta-binomial model

Let us now derive the posterior distribution of this model. The posterior distribution is $\pi(p|x) \propto \pi(x|p)\pi(p)$, which means:

$$\text{Likelihood: } \pi(x|p) = \prod_{i=1}^n \text{bin}(x_i; m, p) = \prod_{i=1}^n \binom{m}{x_i} p^{x_i} (1-p)^{m-x_i} = p^{\sum_{i=1}^n x_i} (1-p)^{nm - \sum_{i=1}^n x_i}$$

$$\text{Prior: } \pi(p|\alpha_0, \beta_0) = \frac{p^{\alpha_0-1} (1-p)^{\beta_0-1}}{B(\alpha_0, \beta_0)}$$

Combining the two distribution, we have:

$$\pi(p|x) \propto \pi(x|p)\pi(p) = p^{\sum_{i=1}^n x_i} (1-p)^{nm - \sum_{i=1}^n x_i} \prod_{i=1}^n \binom{m}{x_i} \times \frac{p^{\alpha_0-1} (1-p)^{\beta_0-1}}{B(\alpha_0, \beta_0)}$$

Because $\prod_{i=1}^n \binom{m}{x_i}$ and $B(\alpha_0, \beta_0)$ do not depend on p , they are constants. Thus,

$$\pi(p|x) \propto p^{\sum_{i=1}^n x_i + \alpha_0 - 1} (1-p)^{nm - \sum_{i=1}^n x_i + \beta_0 - 1}$$

and finally,

$$\pi(p|x) \propto p^{\sum_{i=1}^n x_i + \alpha_0 - 1} (1-p)^{nm - \sum_{i=1}^n x_i + \beta_0 - 1},$$

which is the beta distribution with parameters $\alpha_n = \sum_{i=1}^n x_i + \alpha_0$ and $\beta_n = nm - \sum_{i=1}^n x_i + \beta_0$.

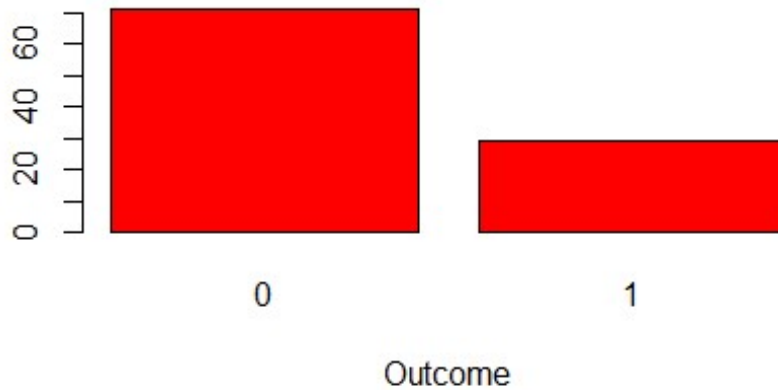
7. Example for the Bernoulli population

To illustrate this procedure, we draw 100 observations from a Bernoulli population with $p = 0.3$. The 100 outcomes are shown on the next plot.

```
# Simulate Bernoulli distribution
```

```
set.seed(123)
p_success <- 0.3
bernoulli_data <- rbinom(100, size = 1, prob = p_success)
barplot(table(bernoulli_data),
        main = "Sample from the Bernoulli population (p=0.3)",
        xlab = "Outcome", col = "red", border = "black")
```

Sample from the Bernoulli population (p=0.3)



Let us compute the parameters of the posterior distribution. For $m = 1$ (the Bernoulli distribution), we have: $\alpha_n = \sum_{i=1}^n x_i + \alpha_0$ and $\beta_n = n - \sum_{i=1}^n x_i + \beta_0$. If we assume that the hyperparameters are $\alpha_0 = 1$ and $\beta_0 = 1$ (uniform prior), then the posterior parameters are:

```
# Sample size
```

```
n <- length(bernoulli_data)
```

```
# Prior hyperparameters
```

```
alpha_0 <- 1
```

```
beta_0 <- 1
```

```
# Calculate posterior parameters
```

```
alpha_n <- sum(bernoulli_data) + alpha_0
```

```
beta_n <- n - sum(bernoulli_data) + beta_0
```

```
# summary
```

```
cat("Posterior alpha:", alpha_n, "\n")
```

```
Posterior alpha: 30
```

```
cat("Posterior beta:", beta_n, "\n")
```

```
Posterior beta: 72
```

```
cat("Posterior mean:", alpha_n/(alpha_n+beta_n), "\n") #Mean
```

```
Posterior mean: 0.2941176
```


The graph below shows the prior and posterior distributions as well as the true value of the parameter $p = 0.3$.

```
library(ggplot2)

# Prior distribution

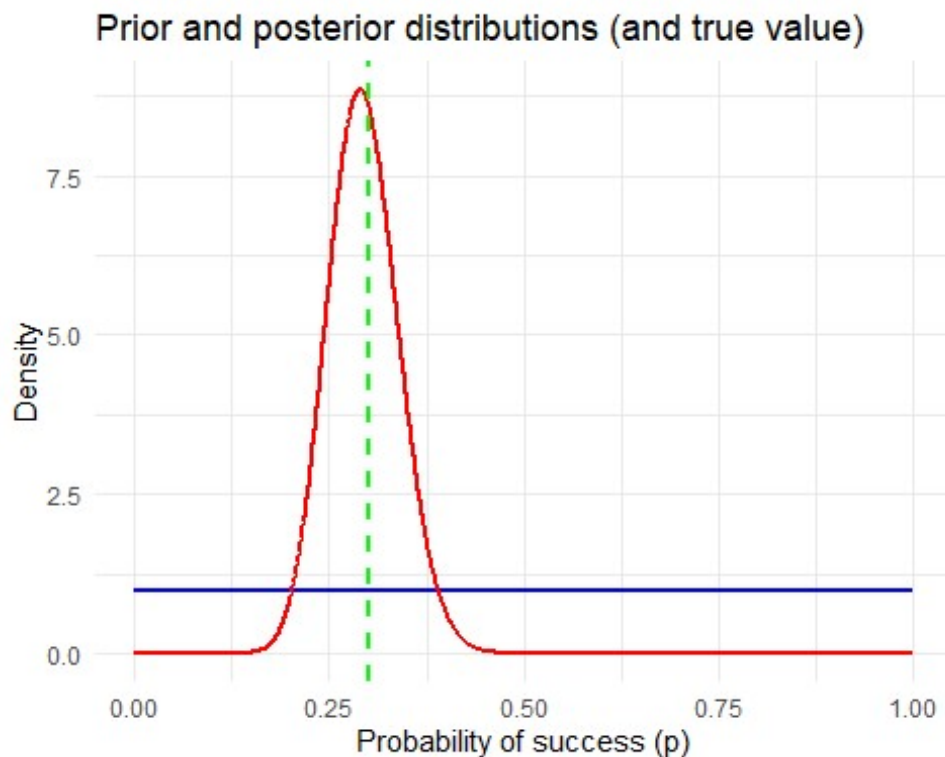
xx    <- seq(0, 1, length.out = 1000)
prior_density <- dbeta(xx, alpha_0, beta_0)

# Posterior distribution

posterior_density <- dbeta(xx, alpha_n, beta_n)

# Plot prior and posterior distributions

p <- ggplot() +
  geom_line(aes(x = xx, y = prior_density),
    color = "blue", linewidth = 1, show.legend=TRUE) +
  geom_line(aes(x = xx, y = posterior_density),
    color = "red", linewidth = 1, show.legend=TRUE) +
  geom_vline(xintercept = p_success, linetype = "dashed",
    color = "green", linewidth = 1, show.legend=TRUE) +
  scale_color_manual(values = c("blue", "red", "green"),
    name = "Distribution",
    labels = c("Prior", "Posterior", "True value")) +
  theme_minimal() +
  labs(title = "Prior and posterior distributions (and true value)",
    x = "Probability of success (p)",
    y = "Density") +
  theme(legend.position = "top")
p
```



8. Our very first example on sampling from the posterior distribution

Now we introduce our very first example of the use of sampling as an approximation to the posterior distribution. If we know the posterior, we can infer everything about the parameter.

```
#install.packages('rstan')
```

```
library(rstan)
```

Loading required package: StanHeaders

rstan version 2.32.6 (Stan version 2.32.2)

For execution on a local, multicore CPU with excess RAM we recommend calling `options(mc.cores = parallel::detectCores())`.

To avoid recompilation of unchanged Stan programs, we recommend calling `rstan_options(auto_write = TRUE)`

For within-chain threading using ``reduce_sum()`` or ``map_rect()`` Stan functions,

change ``threads_per_chain`` option:

```
rstan_options(threads_per_chain = 1)
```

Do not specify `'-march=native'` in `'LOCAL_CPPFLAGS'` or a Makevars file

```
# Stan model code
```

```
stan_model_code <- "  
data {  
  int<lower=0> n;           // Number of observations  
  int<lower=0, upper=1> y[n]; // Binary outcomes  
}  
  
parameters {  
  real<lower=0, upper=1> p; // Probability of success  
}  
  
model {  
  p ~ beta(1, 1);           // Prior distribution for p  
  y ~ bernoulli(p);         // Likelihood  
}  
"
```

```
# Compile the Stan model
```

```
stan_model <- stan_model(model_code = stan_model_code)
```

```
# Create a list of data for Stan
```

```
stan_data <- list(n = n, y = bernoulli_data)
```

```
# Fit the model to the data
```

```
stan_fit <- sampling(stan_model, data = stan_data, chains = 4, iter = 1000)
```

SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 1).

Chain 1:

Chain 1: Gradient evaluation took 9e-06 seconds

Chain 1: 1000 transitions using 10 leapfrog steps per transition would take 0.09 seconds.

Chain 1: Adjust your expectations accordingly!

Chain 1:

Chain 1:

Chain 1: Iteration: 1 / 1000 [0%] (Warmup)

Chain 1: Iteration: 100 / 1000 [10%] (Warmup)

Chain 1: Iteration: 200 / 1000 [20%] (Warmup)

Chain 1: Iteration: 300 / 1000 [30%] (Warmup)

Chain 1: Iteration: 400 / 1000 [40%] (Warmup)

Chain 1: Iteration: 500 / 1000 [50%] (Warmup)

Chain 1: Iteration: 501 / 1000 [50%] (Sampling)

Chain 1: Iteration: 600 / 1000 [60%] (Sampling)

Chain 1: Iteration: 700 / 1000 [70%] (Sampling)

Chain 1: Iteration: 800 / 1000 [80%] (Sampling)

Chain 1: Iteration: 900 / 1000 [90%] (Sampling)
Chain 1: Iteration: 1000 / 1000 [100%] (Sampling)
Chain 1:
Chain 1: Elapsed Time: 0.003 seconds (Warm-up)
Chain 1: 0.003 seconds (Sampling)
Chain 1: 0.006 seconds (Total)
Chain 1:

SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 2).

Chain 2:
Chain 2: Gradient evaluation took 3e-06 seconds
Chain 2: 1000 transitions using 10 leapfrog steps per transition would take 0.03 seconds.
Chain 2: Adjust your expectations accordingly!
Chain 2:
Chain 2:
Chain 2: Iteration: 1 / 1000 [0%] (Warmup)
Chain 2: Iteration: 100 / 1000 [10%] (Warmup)
Chain 2: Iteration: 200 / 1000 [20%] (Warmup)
Chain 2: Iteration: 300 / 1000 [30%] (Warmup)
Chain 2: Iteration: 400 / 1000 [40%] (Warmup)
Chain 2: Iteration: 500 / 1000 [50%] (Warmup)
Chain 2: Iteration: 501 / 1000 [50%] (Sampling)
Chain 2: Iteration: 600 / 1000 [60%] (Sampling)
Chain 2: Iteration: 700 / 1000 [70%] (Sampling)
Chain 2: Iteration: 800 / 1000 [80%] (Sampling)
Chain 2: Iteration: 900 / 1000 [90%] (Sampling)
Chain 2: Iteration: 1000 / 1000 [100%] (Sampling)
Chain 2:
Chain 2: Elapsed Time: 0.006 seconds (Warm-up)
Chain 2: 0.006 seconds (Sampling)
Chain 2: 0.012 seconds (Total)
Chain 2:

SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 3).

Chain 3:
Chain 3: Gradient evaluation took 4e-06 seconds
Chain 3: 1000 transitions using 10 leapfrog steps per transition would take 0.04 seconds.
Chain 3: Adjust your expectations accordingly!
Chain 3:
Chain 3:
Chain 3: Iteration: 1 / 1000 [0%] (Warmup)
Chain 3: Iteration: 100 / 1000 [10%] (Warmup)
Chain 3: Iteration: 200 / 1000 [20%] (Warmup)
Chain 3: Iteration: 300 / 1000 [30%] (Warmup)
Chain 3: Iteration: 400 / 1000 [40%] (Warmup)
Chain 3: Iteration: 500 / 1000 [50%] (Warmup)
Chain 3: Iteration: 501 / 1000 [50%] (Sampling)
Chain 3: Iteration: 600 / 1000 [60%] (Sampling)

```
Chain 3: Iteration: 700 / 1000 [ 70%] (Sampling)
Chain 3: Iteration: 800 / 1000 [ 80%] (Sampling)
Chain 3: Iteration: 900 / 1000 [ 90%] (Sampling)
Chain 3: Iteration: 1000 / 1000 [100%] (Sampling)
Chain 3:
Chain 3: Elapsed Time: 0.007 seconds (Warm-up)
Chain 3:                0.005 seconds (Sampling)
Chain 3:                0.012 seconds (Total)
Chain 3:
```

SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 4).

```
Chain 4:
Chain 4: Gradient evaluation took 4e-06 seconds
Chain 4: 1000 transitions using 10 leapfrog steps per transition would take
0.04 seconds.
Chain 4: Adjust your expectations accordingly!
Chain 4:
Chain 4:
Chain 4: Iteration:   1 / 1000 [  0%] (Warmup)
Chain 4: Iteration: 100 / 1000 [ 10%] (Warmup)
Chain 4: Iteration: 200 / 1000 [ 20%] (Warmup)
Chain 4: Iteration: 300 / 1000 [ 30%] (Warmup)
Chain 4: Iteration: 400 / 1000 [ 40%] (Warmup)
Chain 4: Iteration: 500 / 1000 [ 50%] (Warmup)
Chain 4: Iteration: 501 / 1000 [ 50%] (Sampling)
Chain 4: Iteration: 600 / 1000 [ 60%] (Sampling)
Chain 4: Iteration: 700 / 1000 [ 70%] (Sampling)
Chain 4: Iteration: 800 / 1000 [ 80%] (Sampling)
Chain 4: Iteration: 900 / 1000 [ 90%] (Sampling)
Chain 4: Iteration: 1000 / 1000 [100%] (Sampling)
Chain 4:
Chain 4: Elapsed Time: 0.005 seconds (Warm-up)
Chain 4:                0.003 seconds (Sampling)
Chain 4:                0.008 seconds (Total)
Chain 4:
```

Now, we have draws from the posterior distribution (1000). Next graph depicts the last 500 draws:

```
# Install and load required libraries
```

```
#install.packages("bayesplot")
library(bayesplot)
```

This is bayesplot version 1.11.1

- Online documentation and vignettes at mc-stan.org/bayesplot
- bayesplot theme set to `bayesplot::theme_default()`

* Does `_not_` affect other ggplot2 plots

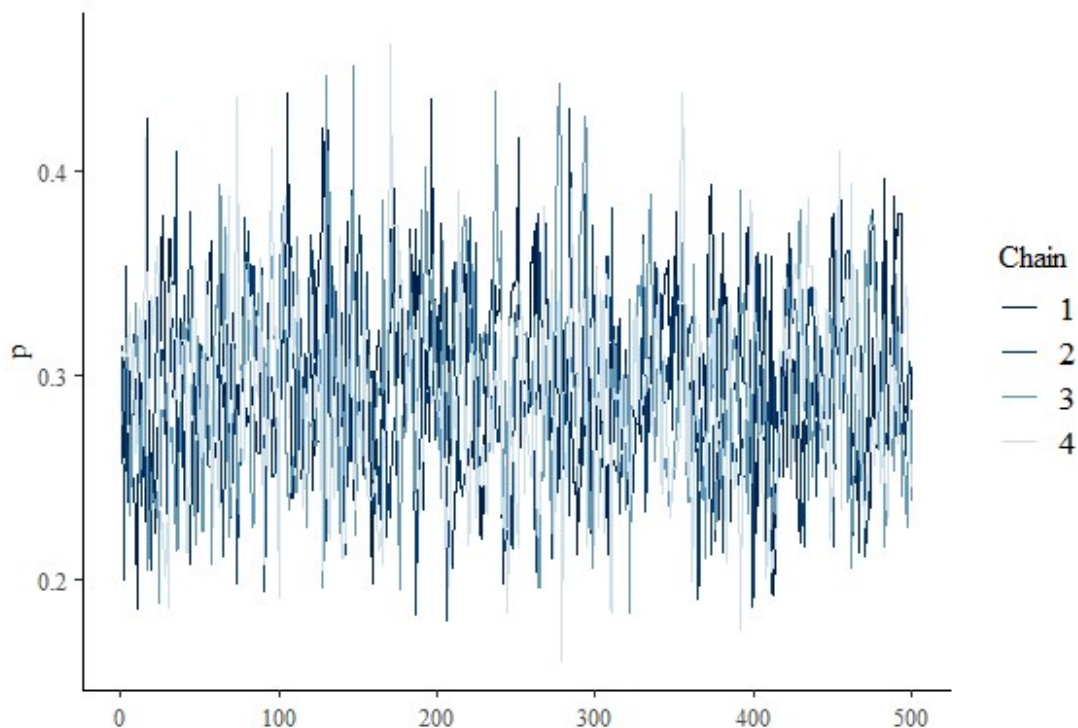
* See ?bayesplot_theme_set for details on theme setting

Extract posterior draws from stan_fit object

```
stan_fit_p <- extract(stan_fit, permuted = FALSE)
```

Traceplot

```
mcmc_trace(stan_fit_p, pars = "p") +  
  theme_default()
```



After analyzing these output (more in next classes), we have the summary of the results.

Print a summary of the Stan fit

```
print(stan_fit)
```

Inference for Stan model: anon_model.

4 chains, each with iter=1000; warmup=500; thin=1;

post-warmup draws per chain=500, total post-warmup draws=2000.

	mean	se_mean	sd	2.5%	25%	50%	75%	97.5%	n_eff	Rhat
p	0.29	0.00	0.05	0.21	0.26	0.29	0.32	0.38	739	1
lp__	-62.32	0.02	0.73	-64.51	-62.50	-62.04	-61.85	-61.79	1145	1

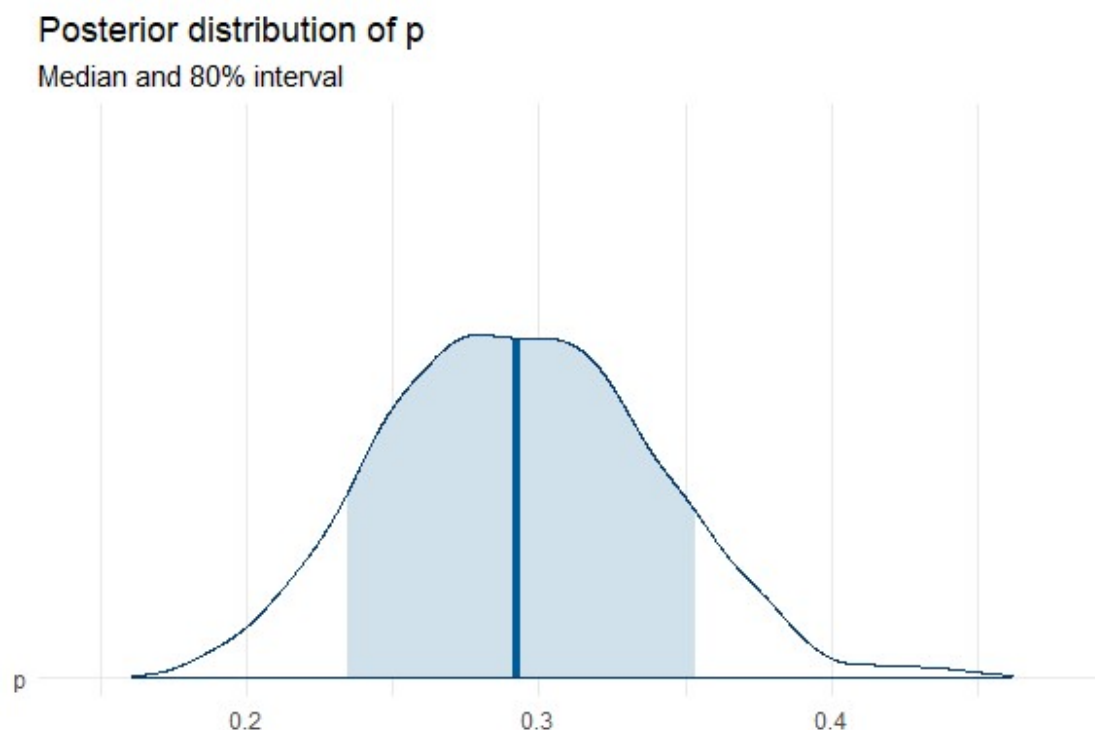
Samples were drawn using NUTS(diag_e) at Thu Feb 20 01:38:27 2025.

For each parameter, n_eff is a crude measure of effective sample size,

and Rhat is the potential scale reduction factor on split chains (at convergence, Rhat=1).

Figure below depicts the density of the posterior distribution based on these draws:

```
# Plotting the posterior distribution
plot_title <- ggtitle("Posterior distribution of p",
"Median and 80% interval")
mcmc_areas(stan_fit, pars = "p", prob = 0.8) +
  plot_title +
  theme_minimal()
```



Finally, we can compute the mean and median of the posterior distribution:

```
# Compare these results
```

```
cat("p estimate (Mean):", mean(stan_fit_p[, , 1]), "\n")
```

```
p estimate (Mean): 0.2932569
```

```
cat("p estimate (Median):", median(stan_fit_p[, , 1]), "\n")
```

```
p estimate (Median): 0.2926115
```

Because the uniform prior is uninformative, the results are quite similar to the MLE. We can make variations on the prior hyperparameters (α_0 and β_0) and sample size (n) to check their impact on the posterior distribution.

9. The logit and probit transformations

The parameter p , probability of success, varies between 0 and 1. Quite often, it is useful to transform those values into real numbers without bounds.

The logistic function is given by

$$y = f(x) = \frac{\exp(x)}{1 + \exp(x)}$$

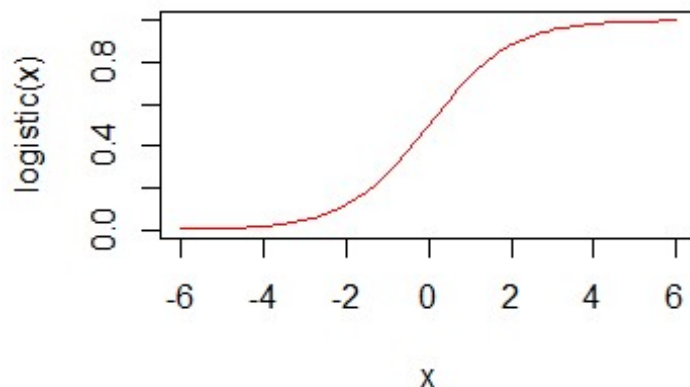
and the plot is

```
# Define the logistic function
```

```
logistic <- function(x) {  
  exp(x)/(1+exp(x))  
}
```

```
# logistic curve
```

```
curve(logistic, -6, 6, col = "red")
```



Two well-know transformations (link functions) project values between 0 and 1 (probabilities, proportions) into the unbounded real number line:

1. the logit transformation: $\eta = \log \frac{p}{1-p}$
2. the probit transformation: $\eta = \Phi^{-1}(p)$

This functions can be seen as the inverse of a distribution and are depicted in the figure below.

```
# Generate a sequence of values between 0 and 1
```

```
xx <- seq(0.001, 0.999, length.out = 100)
```



```

# Logit transformation function
logit <- function(x) {
  log(x / (1 - x))
}

# Probit transformation function
probit <- function(x) {
  qnorm(x)
}

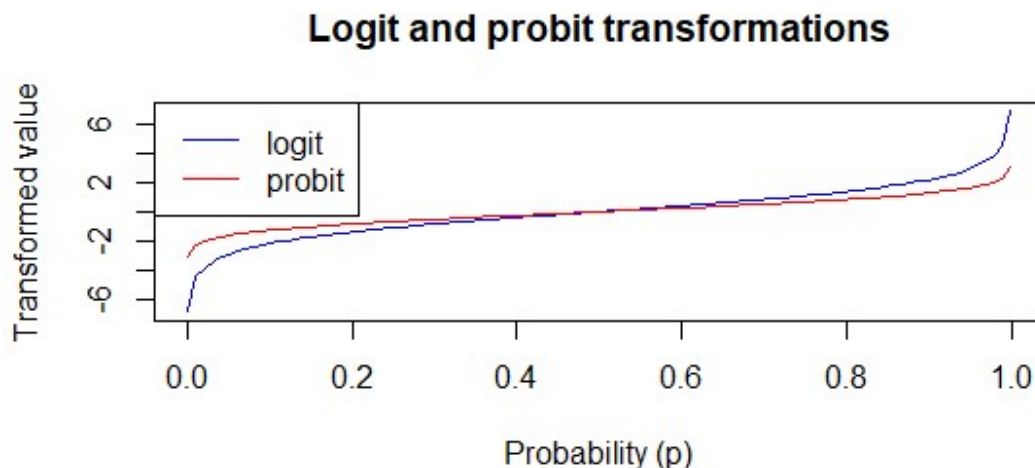
# Apply the transformations to the sequence of values
yy_logit <- logit(xx)
yy_probit <- probit(xx)

# Plot Logit transformation
plot(xx, yy_logit, type = "l", col = "blue",
     main = "Logit and probit transformations",
     xlab = "Probability (p)", ylab = "Transformed value",
     ylim = c(min(yy_logit, yy_probit), max(yy_logit, yy_probit)))

# Add probit transformation to the plot
lines(xx, yy_probit, col = "red")

# Add Legend
legend("topleft", legend = c("logit", "probit"), col = c("blue", "red"), lty
      = 1)

```



The inverse transformations – $p = \frac{\exp(\eta)}{1+\exp(\eta)}$ (logit) and $p = \Phi(\eta)$ (probit) – can be used to predict the probability. Quite often we use these transformations to facilitate the modeling

of nonlinear relation (GLM model to be discussed in next classes). Thus, the next graph shows the predicted probability for a given value η .

```
# Generate a sequence of values between 0 and 1
xx <- seq(-5, 5, length.out = 100)

# Inverse Logit transformation function
invlogit <- function(x) {
  exp(x)/(1+exp(x))
}

# Probit transformation function
invprobit <- function(x) {
  pnorm(x)
}

# Apply the transformations to the sequence of values
yy_logit <- invlogit(xx)
yy_probit <- invprobit(xx)

# Plot inverse Logit transformation
plot(xx, yy_logit, type = "l", col = "blue",
      main = "Inverse logit and probit transformations",
      ylab = "Probability (p)", xlab = "",
      ylim = c(min(yy_logit, yy_probit), max(yy_logit, yy_probit)))

# Add invserse probit transformation to the plot
lines(xx, yy_probit, col = "red")

# Add Legend
legend("topleft", legend = c("inverse logit", "inverse probit"), col =
c("blue", "red"), lty = 1)
```

